

Customer Persona Resolution for Unified Customer Data

Design and operation of an AI-native Persona Engine based on the current schema and source code

Trieu Nguyen

2026-08-04

Abstract

Customer Persona Resolution (CPR) transforms Master Profiles into AI-native Personas. Each persona is a multidimensional representation built from six scoring components: behavior, engagement, financial, loyalty, relationship, and risk. The system can generate readable non-PII persona names with an LLM, produce 768-dimensional embeddings for lookalike discovery, classify risk into four levels (critical/high/medium/low), and classify value into four tiers (champion/high-value/growth-potential/at-risk). A versioning and audit mechanism records material changes (delta score ≥ 5), which supports compliance, trend analysis, and anomaly detection.

1. Scope

This document is grounded in components that already exist in the repository:

- PostgreSQL schema and Persona tables in `database-init/database-schema.sql`
- Persona engine module in `backend-system/identity_resolution/identity_resolution/persona_engine.py`
- Persona computation logic in `backend-system/identity_resolution/identity_resolution/persona.py`
- `PersonaResolutionEngine` and `PersonaVersioningManager` in `resolver.py`
- Vector support through `pgvector` on PostgreSQL 16

Main Flow (Persona Resolution Context)

Raw Profiles

- > Identity Resolution
- > Unified Customer Profile
- > Persona Resolution Engine (this paper)
 - > Feature Engineering
 - > Component Scoring and Aggregation
 - > Risk/Value Classification
 - > LLM Persona Name and Summary
 - > Vector Embedding
 - > Confidence Estimation
- > Customer360 Master Profile (Persona-enriched)

Stage	Purpose	Output
Raw Profiles	Collect multi-source customer signals	Staging records
Identity Resolution	Merge records by matching rules	Unified customer profile
Persona Resolution	Compute score, tier, risk, narrative, and embedding	Persona-enriched profile
Customer360 Master Profile	Serve downstream activation and analytics	Single operational customer view

2. Core Data Model and Scoring Structure

The database stores Persona entities and supporting relations required for end-to-end customer evaluation. The following tables are central.

2.1 Table cdp_customer_personas

This is the Golden Persona record. It stores a full snapshot per customer, including persona identity fields, six component scores, aggregated persona_score (0-100), embedding vector, risk level, value tier, and confidence.

Key fields:

- persona_id, master_profile_id, tenant_id, domain (scoping and keys)
- persona_name, persona_summary (LLM-generated, non-PII)
- persona_score = (0.20 * behavior) + (0.20 * engagement) + (0.20 * financial) + (0.15 * loyalty) + (0.10 * relationship) + (0.15 * (100 - risk))
- persona_embedding (768-dimensional pgvector, cosine similarity use case)
- computed_version (increments on material score change), confidence_score (0-1)
- risk_level and value_tier
- is_active, computed_at, updated_at

2.2 Table cdp_persona_features

Feature store for raw and derived profile features used in scoring, model behavior, and audit traceability.

Key fields: feature_id (PK), persona_id (FK), feature_name, feature_value (NUMERIC), feature_group (behavior/engagement/financial/loyalty/relationship/risk), computed_at

2.3 Table cdp_persona_score_details

Component-level breakdown for explainability and audit.

Key fields: score_detail_id (PK), persona_id (FK), score_component, component_value (0-100), component_weight, weighted_contribution, bonuses_applied (JSONB), computed_at

2.4 Table cdp_persona_history

History of material changes (delta score ≥ 5) for trend tracking, anomaly review, and compliance.

Key fields: history_id (PK), persona_id (FK), old/new scores, score_delta, old/new lifecycle stage, old/new value tier, changed_components, change_reason, recorded_at

2.5 Table cdp_persona_config

Runtime registry for scoring configuration values such as thresholds, weights, bonuses, and caps.

Key fields: config_key (PK), config_value, config_description, data_type, is_active, updated_by, updated_at

3. Persona Scoring Mechanism

The Persona score is computed from six independent components, then aggregated with fixed weights into one bounded score (persona_score in [0, 100]).

3.1 Component 1: Behavior Score

Formula: behavior_score = lifecycle_base + segment_bonus + kyc_bonus + channel_bonus (capped at 100)

Lifecycle Stage	Base Points	Segment	Segment Bonus
PROSPECT	20	low	0
LEAD	40	medium	5
CUSTOMER	65	high	10
VIP	95	critical	15

KYC Status	KYC Bonus	Channel Count	Channel Bonus
unverified	0	1 channel	+10
pending	5	2 channels	+20
verified	15	3+ channels	+30 (max)
rejected	0	n/a	n/a

3.2 Component 2: Engagement Score

Formula: $\text{engagement_score} = \text{recency_score} + (\text{channel_bonus} \times 0.7)$ (capped at 100)

Days Since Activity	Score	Category
≤ 7	100	RECENT_7D
8-30	80	RECENT_30D
31-90	50	RECENT_90D
91-180	25	RECENT_180D
> 180	10	STALE

3.3 Component 3: Financial Score

Financial score reflects customer value using customer lifetime value (CLV), with predictive CLV preferred when higher.

Formula:

$\text{financial_score} = \min(100, (\text{CLV} / \text{CLV_reference}) \times \text{financial_score_multiplier})$

where:

- $\text{CLV} = \max(\text{historical_clv}, \text{predictive_clv})$
- CLV_reference defaults to 5000.0
- $\text{financial_score_multiplier}$ defaults to 100.0

Example:

- $\text{historical_clv} = 8500, \text{predictive_clv} = 12000$
- $\text{CLV} = 12000$
- $\text{financial_score} = \min(100, (12000 / 5000) \times 100) = 100.0$

3.4 Component 4: Loyalty Score

Loyalty score estimates long-term relationship strength through membership tier and customer tenure.

Formula:

$\text{loyalty_score} = \text{tier_base} + \text{tenure_bonus}$

where:

- tier_base by membership_tier :
PLATINUM=100, GOLD=80, SILVER=60, BRONZE=40, DEFAULT=20
- $\text{tenure_bonus} = \min((\text{tenure_days} / 365.0) \times \text{tenure_bonus_per_year}, \text{tenure_bonus_cap})$
- $\text{tenure_bonus_per_year}$ defaults to 20.0
- tenure_bonus_cap defaults to 20.0

Example:

- $\text{membership_tier} = \text{GOLD}, \text{customer_since} = 600 \text{ days ago}$
- $\text{tier_base} = 80$
- $\text{tenure_bonus} = \min((600 / 365) \times 20, 20) = 20$
- $\text{loyalty_score} = 100$

3.5 Component 5: Relationship Score

Relationship score estimates channel breadth and contact depth.

Formula:

`relationship_score = channel_score + contact_score`

where:

- `channel_score = min(number_of_source_systems x 20.0, 60.0)`
- `contact_score = min(number_of_contacts x 10.0, 40.0)`

Example:

- 3 source systems, 2 emails, 2 phones
- `channel_score = min(3 x 20, 60) = 60`
- `contact_score = min(4 x 10, 40) = 40`
- `relationship_score = 100`

3.6 Component 6: Risk Score

Risk score combines churn probability, risk segment, and KYC status.

Formula:

`risk_score = (churn_probability x churn_multiplier) + segment_bonus + kyc_status_bonus`

where:

- `churn_multiplier` defaults to 100.0
- `segment_bonus` depends on `risk_segment`
- `kyc_status_bonus` depends on `kyc_status`
- fallback churn base is used when `churn_probability` is null

Example:

- `churn_probability = 0.45`, `risk_segment = high`, `kyc_status = verified`
- `risk_score = (0.45 x 100) + 40 + 0 = 85`

3.7 Composite Persona Score

Main formula:

`persona_score = (behavior_score * weight_behavior)`
`+ (engagement_score * weight_engagement)`
`+ (financial_score * weight_financial)`
`+ (loyalty_score * weight_loyalty)`
`+ (relationship_score * weight_relationship)`
`+ ((100 - risk_score) * weight_risk)`

Weights used by default:

- behavior: 0.20
- engagement: 0.20
- financial: 0.20
- loyalty: 0.15
- relationship: 0.10
- risk inverse term: 0.15

Final score is clamped into [0, 100].

Worked example:

<code>behavior_score=75.0,</code>	<code>75.0 x 0.20 = 15.0</code>
<code>engagement_score=85.0,</code>	<code>85.0 x 0.20 = 17.0</code>
<code>financial_score=90.0,</code>	<code>90.0 x 0.20 = 18.0</code>
<code>loyalty_score=80.0,</code>	<code>80.0 x 0.15 = 12.0</code>
<code>relationship_score=70.0,</code>	<code>70.0 x 0.10 = 7.0</code>
<code>risk_score=40.0,</code>	<code>(100-40) x 0.15 = 9.0</code>

`persona_score = 78.0`

4. Risk and Value Classification

4.1 Risk Level Binning

risk_score is converted from a numeric score to a business category for downstream decisioning.

```
def compute_risk_level(risk_score):  
    if risk_score >= RISK_LEVEL_CRITICAL_THRESHOLD: # 80.0  
        return "critical"  
    elif risk_score >= RISK_LEVEL_HIGH_THRESHOLD: # 60.0  
        return "high"  
    elif risk_score >= RISK_LEVEL_MEDIUM_THRESHOLD: # 40.0  
        return "medium"  
    return "low"
```

Risk Level	Threshold	Meaning	Typical Business Action
critical	>= 80	Very high risk	Reject or heavily constrain, manual approval required
high	60-79	High risk	Conditional approval, tighter monitoring
medium	40-59	Moderate risk	Standard approval with periodic checks
low	< 40	Low risk	Fast-track approval and growth offers

4.1.1 Use Case Examples

Scenario	Risk Score	Risk Level	Decision	Action
Churn 72%, fraud flags, unverified KYC	92.0	critical	Reject offer	Manual review required
Churn 35%, high segment, dormant 4 months	68.0	high	Conditional approval	Supervisor review and re-engagement offer
Churn 20%, medium segment, pending KYC	48.0	medium	Approve with monitoring	Standard processing with watchlist
Churn 5%, verified KYC, long tenure, high CLV	22.0	low	Priority approval	VIP treatment and upsell eligibility

4.1.2 Risk Decision Flow (Simplified)

Risk Level	Score Rule	Checks	Decision
critical	risk_score >= 80	Fraud indicators and compliance watchlists	Reject/limit and require manual approval
high	60 <= risk_score < 80	Engagement drop and recent transaction patterns	Conditional approval (rate/limit controls)
medium	40 <= risk_score < 60	Standard profile review	Standard approval with periodic monitoring
low	risk_score < 40	Credit and growth potential check	Priority approval and upsell recommendations

4.1.3 Example Risk Distribution Benchmark

The following distribution is an illustrative benchmark on a 700-customer retail set.

Risk Tier	Range	Count	Share	Average Persona Score
critical	>= 80.0	35	5.0%	31.4
high	60.0-79.9	140	20.0%	47.8
medium	40.0-59.9	380	54.3%	62.5
low	< 40.0	145	20.7%	79.2

Note: Correlation between churn probability and risk score is approximately 0.87 in this benchmark.

4.2 Customer Value Tier Classification

The system maps `persona_score` into value tiers for personalization and resource allocation.

```
def compute_customer_value_tier(persona_score):
    if persona_score >= VALUE_TIER_CHAMPION_THRESHOLD:           # 80.0
        return "champion"
    elif persona_score >= VALUE_TIER_HIGH_VALUE_THRESHOLD:       # 60.0
        return "high_value"
    elif persona_score >= VALUE_TIER_GROWTH_POTENTIAL_THRESHOLD: # 35.0
        return "growth_potential"
    return "at_risk"
```

Value Tier	Threshold	Meaning	Typical Strategy
champion	>= 80	Highest value and loyalty	VIP treatment, retention-first, aggressive cross-sell
high_value	60-79	Strong value with growth upside	Tier upgrade and strategic cross-sell
growth_potential	35-59	Developable potential	Nurture campaigns and education flows
at_risk	< 35	Low value or churn-prone	Win-back and offer optimization

Examples:

- `persona_score = 88.0 -> champion`
- `persona_score = 70.0 -> high_value`
- `persona_score = 45.0 -> growth_potential`
- `persona_score = 25.0 -> at_risk`

5. LLM-native Persona Name and Summary Generation

5.1 Persona Name Synthesis

Each customer receives a non-PII `persona_name` that is readable in dashboards and safe for broad operational use.

Primary goals:

- Human-friendly display in UI and analytics
- PII protection, especially for hashed-profile paths
- Better narrative quality for campaign and CX teams

Generation flow:

1. Ingest `master_profile`.
2. Detect whether profile identity appears hashed.
3. If LLM is available, generate a 5-7 word non-PII persona name.
4. If API key is missing or LLM call fails, use deterministic fallback from identity anchors.

5. Persist final persona_name in cdp_customer_personas.

LLM prompt template (Gemini 3.5 Flash):

Generate a memorable, non-PII persona name (5-7 words) for a customer based on:

- Domain: {domain}
- Lifecycle Stage: {lifecycle_stage}
- Primary Channel: {preferred_channel}
- Engagement Level: {engagement_score}
- Financial Value: {financial_score}

Persona Name:

Deterministic fallback:

```
if is_hashed:
    anchor_id = first_non_null(device_id, advertising_id, cookie_id)
    hash_suffix = sha256(anchor_id)[:6].lower()
    persona_name = f"Digital Customer #{hash_suffix}"
else:
    persona_name = f"{full_name} (Customer)"
```

5.2 Persona Summary

The system generates a short 2-3 sentence summary from behavior, engagement, loyalty tier, risk level, value tier, and channel footprint.

Example summary:

"Premium customer with strong cross-channel engagement, Gold-tier loyalty, and low churn risk."

6. Vector Embeddings and Lookalike Discovery

6.1 Persona Embedding (768 dimensions)

persona_embedding is produced from persona text and key features to support:

- Semantic search over customers
- Lookalike audience discovery
- Outlier and anomaly analysis
- Clustering and segmentation

Embedding flow:

1. Compose text from persona_name, persona_summary, and selected features.
2. Call Gemini Embeddings API.
3. Normalize output into pgvector-compatible format.
4. Store in cdp_customer_personas.persona_embedding.

Feature text example:

```
feature_text = f"""
Customer Profile:
- Behavior: {behavior_score}/100 ({lifecycle_stage})
- Engagement: {engagement_score}/100 (last activity {recency_days} days ago)
- Financial: {financial_score}/100 (CLV ${clv})
- Loyalty: {loyalty_score}/100 ({membership_tier})
- Relationship: {relationship_score}/100 ({len(source_systems)} channels)
- Risk: {risk_score}/100 ({risk_level}, churn prob {churn_probability:.2%})
- Value Tier: {value_tier}
"""
```

6.2 Lookalike Audience Discovery

A typical query uses cosine similarity from pgvector.

```

SELECT
    persona_id,
    persona_name,
    (1 - (persona_embedding <=> ref_embedding)) AS sim
FROM cdp_customer_personas
WHERE (1 - (persona_embedding <=> ref_embedding)) > 0.75
ORDER BY sim DESC
LIMIT 100;

```

Use case: find lookalike customers for a high-performing persona segment in cross-sell campaigns.

7. Persona Versioning and Audit Trail

7.1 Versioning Logic

Each persona has a monotonically increasing `computed_version`.

Version increments only when:

- $\text{abs}(\text{new_persona_score} - \text{old_persona_score}) \geq \text{PERSONA_HISTORY_SCORE_DELTA_THRESHOLD}$
- default threshold is 5.0 points

This prevents audit noise from small, non-material changes.

Example sequence:

- V1: score = 75.0
- Candidate update: score = 78.0, delta = 3.0 -> no new version
- Candidate update: score = 82.0, delta = 4.0 -> no new version
- Candidate update: score = 81.5, delta = 6.5 -> new version + history record

7.2 Audit Record Structure

When a material change occurs, the engine writes a row to `cdp_persona_history`.

```

if abs(new_score - old_score) >= PERSONA_HISTORY_SCORE_DELTA_THRESHOLD:
    history_record = {
        "persona_id": persona.persona_id,
        "old_persona_score": old_score,
        "new_persona_score": new_score,
        "score_delta": new_score - old_score,
        "old_lifecycle_stage": old_lifecycle_stage,
        "new_lifecycle_stage": new_lifecycle_stage,
        "old_value_tier": old_value_tier,
        "new_value_tier": new_value_tier,
        "changed_components": ["engagement_score", "risk_score"],
        "change_reason": "recency_update",
        "recorded_at": datetime.now(),
    }

```

8. Confidence Estimation

Each persona receives `confidence_score` in $[0, 1]$.

Formula:

$\text{confidence_score} = \max(\text{identity_confidence_score}, \text{profile_completeness_score})$

where:

- `identity_confidence_score` comes from identity resolution strength
- `profile_completeness_score` is non-null field coverage ratio

Interpretation:

- 0.9: high confidence, suitable for direct operational use

- 0.5: medium confidence, use with additional checks
 - 0.2: low confidence, collect more data first
-

9. Centralized Scoring Configuration

Scoring constants are runtime-controlled by `cdp_persona_config` with safe fallback to `PERSONA_CONFIG_DEFAULTS` in the engine.

Configuration blocks include:

1. Risk thresholds
2. Risk scoring multipliers and bases
3. Lifecycle behavior bases
4. Engagement recency thresholds/scores and channel caps
5. Financial normalization constants
6. Loyalty tier and tenure constants
7. Relationship channel/contact caps
8. Composite score weights
9. Value tier thresholds
10. History material-change threshold

Benefits:

- Single point of change via DB-backed config
 - Typed parsing support (INTEGER, NUMERIC, BOOLEAN)
 - Reliable fallback when config query fails
 - Better auditability for threshold and weight changes
-

10. Persona Processing Pipeline

10.1 Batch Resolution Steps

1. Load active runtime config from `cdp_persona_config` (fallback to defaults if needed).
2. Scan active rows from `cdp_master_profiles`.
3. Compute six component scores.
4. Aggregate and clamp `persona_score`.
5. Derive `risk_level` and `value_tier`.
6. Generate `persona_name` and `persona_summary` (LLM-first, deterministic fallback).
7. Generate `persona_embedding` and compute `confidence_score`.
8. Compare to previous version and decide whether material change threshold is met.
9. Upsert `cdp_customer_personas` and write supporting detail rows.
10. Commit transaction after batch completion.

10.2 Confidence Calculation Reminder

```
confidence_score = max(identity_confidence, profile_completeness)
```

- `identity_confidence`: normalized identifier-match strength
 - `profile_completeness`: normalized populated-field ratio
-

11. Conclusion

The Persona Resolution Engine provides a practical AI-native layer for customer intelligence with strong operational controls:

- six-component scoring for balanced customer evaluation
- LLM-native persona naming and narrative summaries
- vector embeddings for semantic search and lookalike discovery
- threshold-based versioning and audit logs for compliance
- risk and value classification for decision support

In short, the current implementation is technically robust and ready to support personalization, risk-aware operations, and customer growth workflows in a multi-tenant Customer 360 environment.

References: `persona_engine.py` | `database-schema.sql`